

sell sell marketplace updates - 11052026

UPDATE: 11/05/2026:05:04 AM

SHAZ EJ

SellSell + RunPod GPU Transcription — Work Report

May 11, 2026

Objective

Migrate SellSell audio transcription from a local Docker transcription container on the DigitalOcean Next.js server to a dedicated GPU-powered RunPod transcription service using `faster-whisper large-v3` on an NVIDIA L4 GPU.

Goals:

- Offload CPU-heavy transcription from DO server
- Use GPU acceleration for faster Arabic transcription
- Keep Ollama/Qwen running independently
- Expose transcription as an external API
- Integrate seamlessly into SellSell Next.js backend

Infrastructure Setup

RunPod GPU Pod

Configured a persistent RunPod pod with:

Component	Value
GPU	NVIDIA L4
Runtime	CUDA 12.4
Network Storage	/workspace
Model	faster-whisper-large-v3
Ollama	Existing Qwen setup preserved
Exposed Ports	8000, 8888, 11434



Major Debugging Journey

The GPU debugging was the hardest part of the day.

Initial Problem

The pod clearly had an NVIDIA L4 GPU and Ollama was already using it successfully.

However:

- faster-whisper
- ctranslate2
- CUDA Python runtime

all failed to detect any CUDA device.

Errors included:

```
Failed to initialize NVML: Unknown Error
```

and:

```
RuntimeError: CUDA failed with error no CUDA-capable device is detected
```

Root Cause Investigation

We systematically diagnosed the environment.

1. GPU mapped as

```
/dev/nvidia6
```

The container exposed:

```
/dev/nvidia6
```

instead of:

```
/dev/nvidia0
```

Created symlink:

```
ln -sf /dev/nvidia6 /dev/nvidia0
```



This alone did not solve CUDA detection.

2.

`nvidia-smi`

was misleading

`nvidia-smi` failed entirely:

Failed to initialize NVML: Unknown Error

Important discovery:

- RunPod containers often block NVML
- CUDA may still work even if `nvidia-smi` fails

This was a red herring.

3. CUDA device count returned 0

Testing:

```
ctranslate2.get_cuda_device_count()
```

returned:

0

meaning CUDA runtime could not see the GPU.

4. GPU devices mounted read-only

Inspection of mount namespace showed:

```
udev /dev/nvidia6 devtmpfs ro,nosuid,noexec
```

GPU device files were mounted read-only for SSH sessions.

Result:

- SSH shell processes could not open CUDA device files
- Ollama could, because it started earlier in RunPod init chain



5. CUDA initialization failure

Direct CUDA testing:

```
lib.cuInit(0)
```

returned:

```
100 = no CUDA device found
```

confirming CUDA runtime initialization failure.

6. Capability restrictions

Container capabilities showed missing GPU-required permissions:

```
CapEff missing CAP_SYS_ADMIN
```

This explained why manually started SSH processes differed from RunPod-managed startup processes.

7. Discovery: Ollama bundled CUDA runtime

Found Ollama environment:

```
LD_LIBRARY_PATH=/usr/local/lib/ollama:/usr/local/lib/ollama/cuda_v12:/usr/local/nvidia/lib:/usr/local/nvidia/lib64
```

Ollama ships its own CUDA runtime libraries.

Critical insight:

Other GPU applications could reuse Ollama's CUDA stack.

Final GPU Fix

Step 1 — Restart pod

Restarted RunPod pod to reattach GPU device permissions correctly.



Persistent network volume /workspace preserved all data.

Step 2 — Reuse Ollama CUDA libs

Configured runtime environment:

```
export
LD_LIBRARY_PATH=/usr/local/lib/ollama:/usr/local/lib/ollama/cuda_v12:/usr/local/nvidia/lib:/usr/local/nvidia/lib64
```

```
export CUDA_VISIBLE_DEVICES=0
export NVIDIA_VISIBLE_DEVICES=all
export NVIDIA_DRIVER_CAPABILITIES=compute,utility
```

Result:

CUDA devices: 1

GPU became accessible successfully.

Transcription Service Setup

Fixed

main.py

Added missing Uvicorn startup:

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Created persistent startup script

Created:

/workspace/transcription-service/start-gpu.sh

Responsibilities:

- configure CUDA environment
- recreate /post_start.sh
- auto-start transcription service



- persist across pod restarts

Exposed RunPod HTTP Port

Added:

8000

to RunPod exposed HTTP services.

Result:

- public transcription endpoint became available

Final GPU Service State

Component	Status
faster-whisper large-v3	✓ Running
GPU	✓ NVIDIA L4
Compute Type	✓ float16
Port 8000	✓ Public
Ollama/Qwen	✓ Still operational
Health Endpoint	✓ Working

API Endpoints

Health

```
curl https://4yp13edfj14agz-8000.proxy.runpod.net/health
```

Response:

```
{
  "status": "ok",
  "model": "Systran/faster-whisper-large-v3",
  "device": "cuda",
```



```
"compute_type": "float16"
}
```

Transcription

```
curl -X POST \
  https://4yp13edfj14agz-8000.proxy.runpod.net/transcribe \
  -F "audio=@audio.wav"
```

SellSell Next.js Integration

Updated SellSell production architecture.

Modified Files

docker-compose.yml

Prevented transcription container from auto-starting:

```
profiles:
  - local-transcription
```

Result:

- transcription container no longer runs in production
 - still available locally if needed
-

src/app/api/audio/transcribe/route.ts

Removed hardcoded fallback:

```
"http://transcription-service:8001"
```

Now strictly uses:

```
TRANSCRIPTION_SERVICE_URL
```

Also added support for both:

```
text
```

and:

```
transcript
```



response formats.

`.env.example`

Added:

`TRANSCRIPTION_SERVICE_URL=`

Git Operations

Successfully committed and pushed changes:

```
git commit -m "chore(audio): route transcription to RunPod service"
git push origin main
```

Branch state:

`working tree clean`

Local + External Testing

Successfully tested:

- RunPod health endpoint
- SSH tunnel access
- public proxy URL access
- API reachability from local Mac

Verified:

- port 8000 operational
 - GPU service alive
 - external HTTP requests functioning
-

Architecture After Migration

Before

```
DigitalOcean
├── Next.js
├── PostgreSQL
└── Ollama
```



└─ Local CPU transcription container

Problems:

- CPU-heavy
- memory pressure
- slower transcription
- unnecessary local Docker complexity

After

DigitalOcean

└─ Next.js API

RunPod GPU Pod

└─ Ollama/Qwen

└─ faster-whisper GPU service

Benefits:

- GPU acceleration
- cleaner architecture
- lower DO server load
- scalable transcription
- isolated AI infrastructure

Key Learnings

1. RunPod GPU access depends heavily on init chain and mount namespace.
2. `nvidia-smi` failing does NOT imply broken CUDA.
3. Ollama bundles reusable CUDA libraries.
4. `/workspace` persists across pod restarts; `/` does not.
5. RunPod SSH sessions behave differently from startup-managed processes.
6. Docker Compose profiles are useful for keeping optional services local-only.
7. Public RunPod proxy URLs simplify integration dramatically.

Remaining Tasks (Tomorrow)

1. Add production `.env` variable on DigitalOcean:

`TRANSCRIPTION_SERVICE_URL=https://4yp13edfj14agz-8000.proxy.runpod.net`

2. Pull latest Git changes:



git pull

3. Remove old transcription container:

```
docker rm -f Sellsell_transcription
```

4. Restart services:

```
docker compose up -d
```

5. Perform final end-to-end transcription test from SellSell frontend.

Final Outcome

The SellSell platform now has:

- production-ready GPU transcription
- faster-whisper large-v3
- NVIDIA L4 acceleration
- isolated AI infrastructure
- simplified DO deployment
- persistent RunPod AI backend
- external transcription API ready for scaling

The most difficult part — GPU/CUDA access inside RunPod containers — was fully solved successfully.